

B18 Series B

The ecosystem, and a finale: multi-omics, spatial, Python, and what comes next

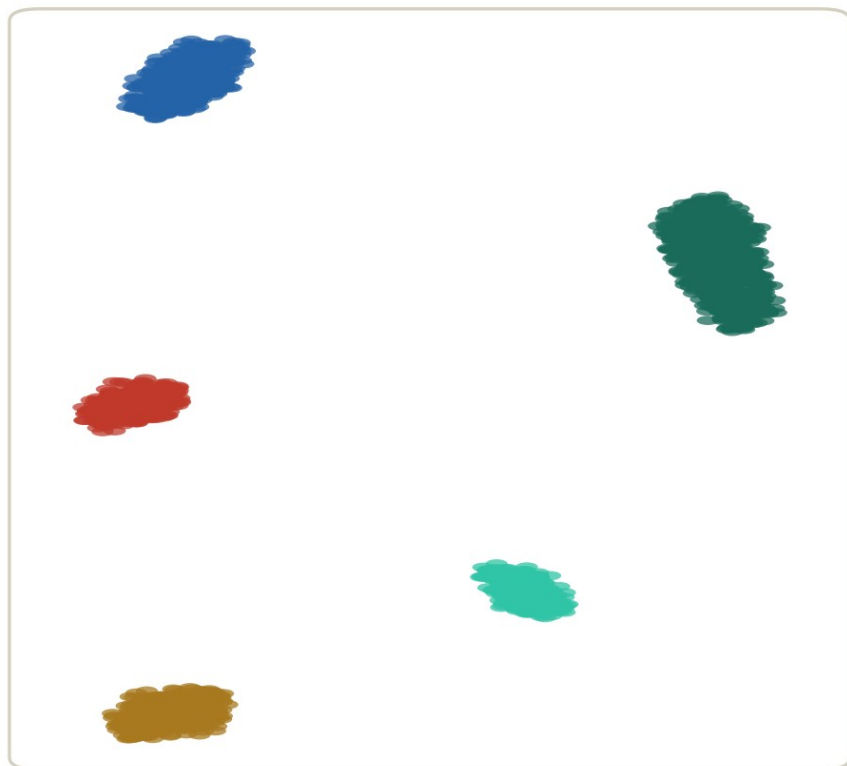
One episode to map the neighbouring technologies, the R/Python ecosystem, and all 19 episodes.

About 35 min | Prerequisites: B1-B17 | Data: existing objects (pbmc3k / ifnb)

Back to the first UMAP

simulated data

B1: your first UMAP



back then: 15 lines, every one copied

now: behind every line, a full episode

Read10X / CreateSeuratObject

B5

subset (QC filter)

B5

NormalizeData

B6

FindVariableFeatures / ScaleData

B6

RunPCA

B7

FindNeighbors / FindClusters

B8

RunUMAP / DimPlot

B9

annotate, integrate, DE, beyond...

B10+

this episode: what lies beyond the map?

B1's fifteen lines — you now know each hides a full episode

THIS EPISODE'S QUESTION

**You have walked the whole map.
What lies beyond its edges?**

RNA is one slice of the cell. This episode lays out the other slices, the other language, and your next step.

What you will learn

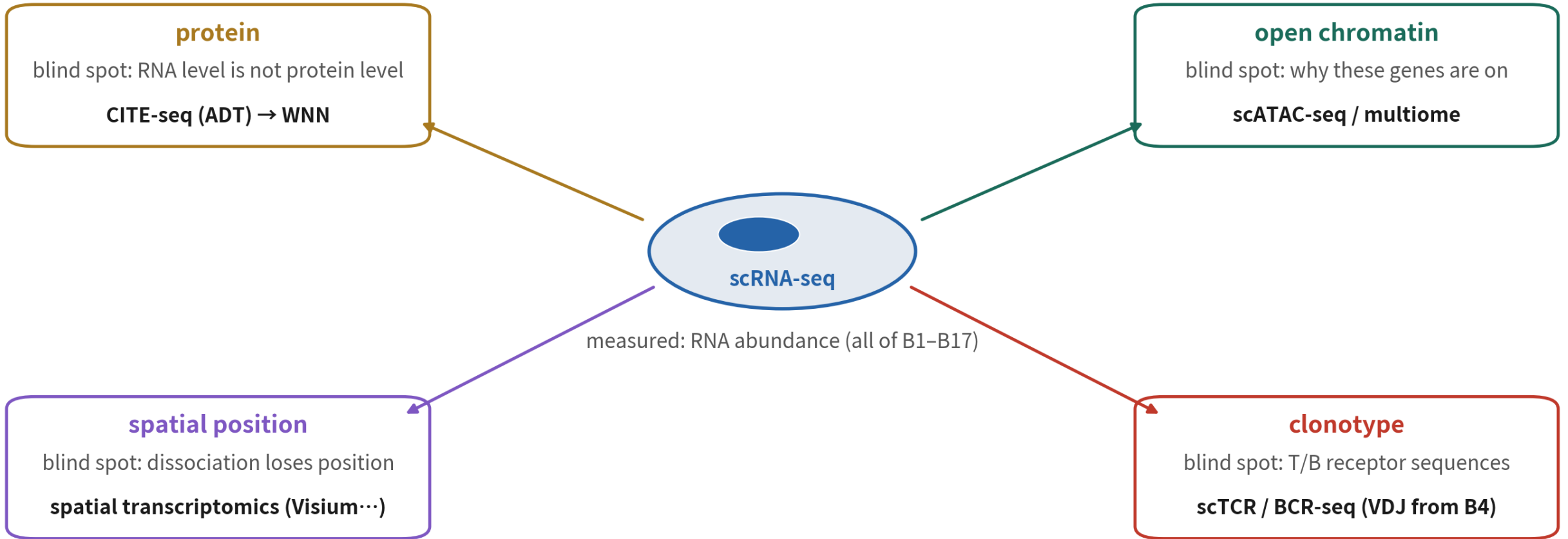
- 1 Name the blind spot each modality patches — protein, chromatin, space, clonotype — and the extra analysis step each brings
- 2 Understand how the R and Python ecosystems divide the work, and convert h5ad files you can verify
- 3 Place yourself on the 19-episode map and leave with a study route

01

Beyond RNA: the modality map

One blind spot, one technology

Four blind spots of the transcriptome



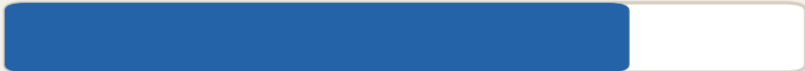
each neighbouring technology patches one blind spot of the transcriptome

Protein, open chromatin, spatial position, clonotype — each has a technology to patch it

The shared idea of multimodal integration: WNN

cell 1: RNA is informative

transcriptome separates it well;
the protein panel lacks its markers

RNA weight  0.78

ADT weight  0.22

weights sum to 1, decided cell by cell

cell 2: protein is informative

naive vs memory T look alike in RNA;
CD45RA/CD45RO protein splits them

RNA weight  0.25

ADT weight  0.75

weights sum to 1, decided cell by cell

the core of WNN: weights are per cell, not per dataset

Each cell carries its own modality weights — trust whichever modality is informative for it

CORE IDEA

**More modalities are not automatically better —
each one answers a different question.**

Ask which slice your question is missing, then decide. If you add one, let methods like WNN give each its proper role.

02

CITE-seq downstream, with WNN

Picking up B4's ADT: two assays analysed together

Two assays, each normalized its own way

```
# 承接 B4 : CITE-seq 讀進來是一個 list, 兩個矩陣
inputs <- Read10X("data/citeseq/filtered_feature_bc_matrix/")
names(inputs)
#> [1] "Gene Expression" "Antibody Capture"

cite <- CreateSeuratObject(counts = inputs[["Gene Expression"]])
cite[["ADT"]] <- CreateAssayObject(
  counts = inputs[["Antibody Capture"]])

cite <- NormalizeData(cite) # RNA : 照 B6 的做法
cite <- NormalizeData(cite, assay = "ADT",
  normalization.method = "CLR", margin = 2)
```

ADT uses CLR with margin = 2 (per cell) — B4 explained why antibodies don't get LogNormalize

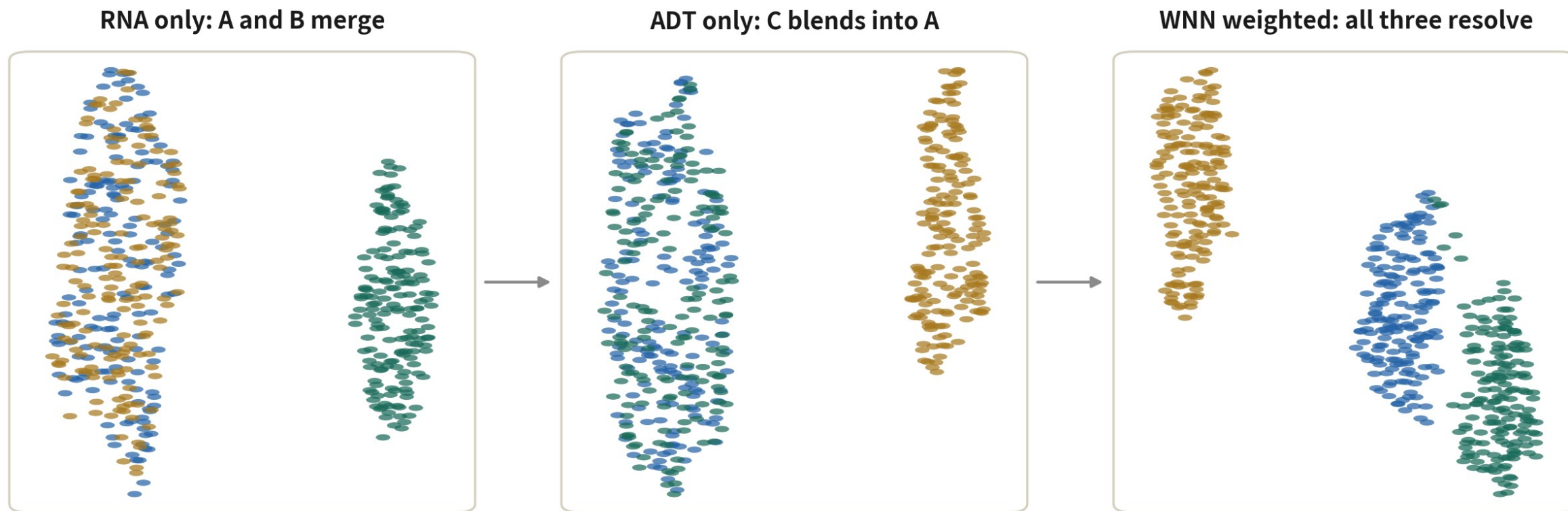
FindMultiModalNeighbors: the key WNN calls

```
# 兩個模態各自降維
DefaultAssay(cite) <- "RNA"
cite <- cite |> FindVariableFeatures() |> ScaleData() |> RunPCA()
DefaultAssay(cite) <- "ADT"
cite <- cite |> ScaleData() |>
  RunPCA(reduction.name = "apca")

# WNN: 兩組 KNN → 每顆細胞的模態權重 → 加權鄰居圖
cite <- FindMultiModalNeighbors(cite,
  reduction.list = list("pca", "apca"),
  dims.list = list(1:30, 1:18))
cite <- RunUMAP(cite, nn.name = "weighted.nn")
cite <- FindClusters(cite, graph.name = "wsnn")
```

Afterwards meta.data gains RNA.weight — each cell's RNA weight, well worth plotting

What WNN does: a simulated demo



computed RNA weights (mean per type): naive T (A) 0.31 memory T (B) 0.24 B cells (C) 0.49

both T subsets hand weight to ADT; B cells pull it back to RNA — weights are computed per cell

simulated data

What RNA cannot split, ADT does — and vice versa. Weights are computed per cell

What this section did

Two assays



RNA and ADT live in one object, each normalized its own way

Two KNN graphs



each modality finds neighbours in its own PC space

One weighted graph



per-cell weights build weighted.nn;
clustering and UMAP proceed as usual

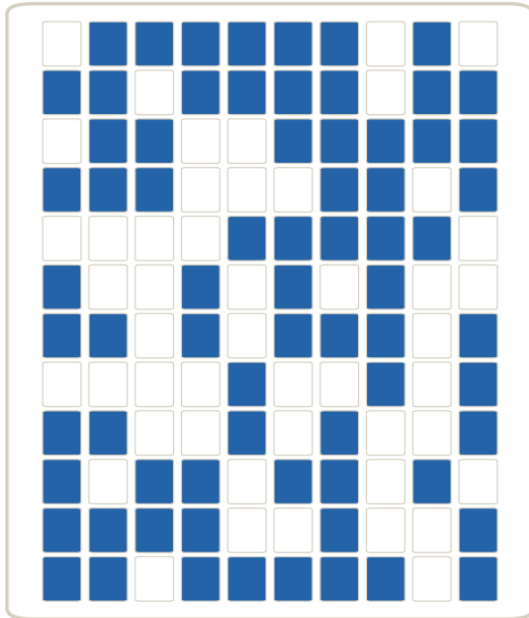
03

scATAC and multiome

One page for peak matrices, one for the bridge

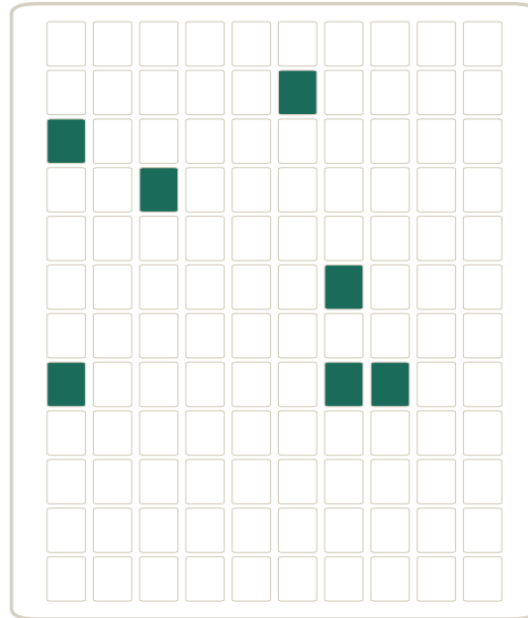
How a peak matrix differs from an RNA matrix

RNA: genes × cells



~20k rows; values 0 to hundreds

ATAC: peaks × cells



100k+ rows; nearly binary

the Seurat workflow carries over, with LSI replacing PCA (see Signac docs)

100k+ features, nearly binary, features are intervals — hence LSI instead of PCA

RNA features: gene names

CD8A · MS4A1 · NKG7 ...

ATAC features: genomic intervals

chr2-87011729-87012986 ...



the bridge: gene activity

sum peaks near each gene body into
a pseudo-expression that talks to RNA

What multiome buys you

Two answers per cell



RNA and ATAC come from the same cell — no cross-dataset guessing

A sense of direction



chromatin opens before RNA appears: regulation gains an order

Tooling: Signac



the Seurat team's ATAC package — objects and syntax you already know

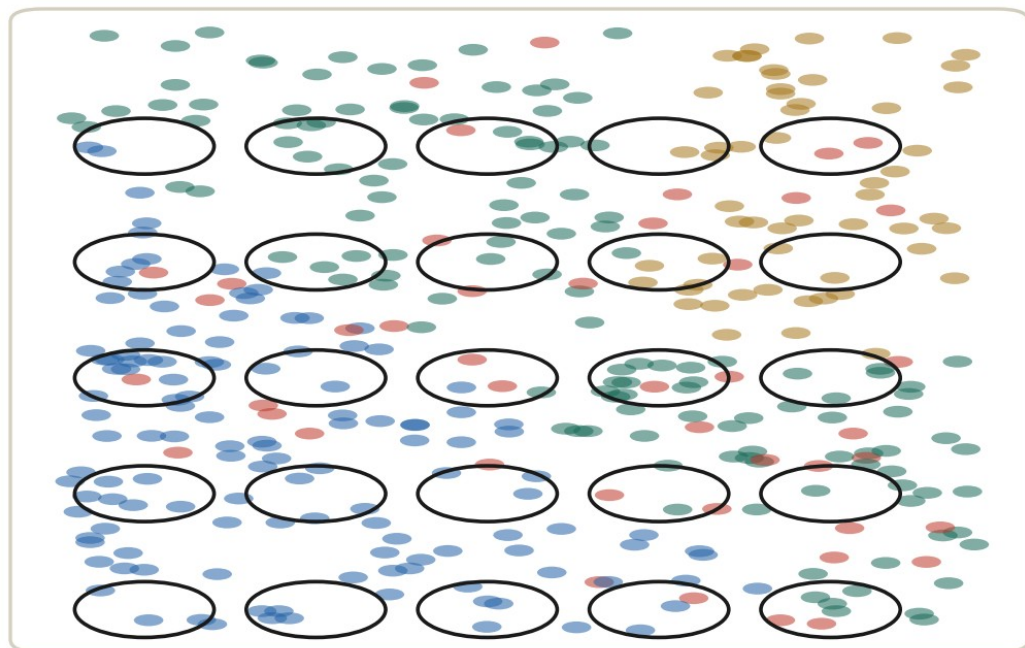
04

Spatial transcriptomics

Position returns — and so does B17's equation

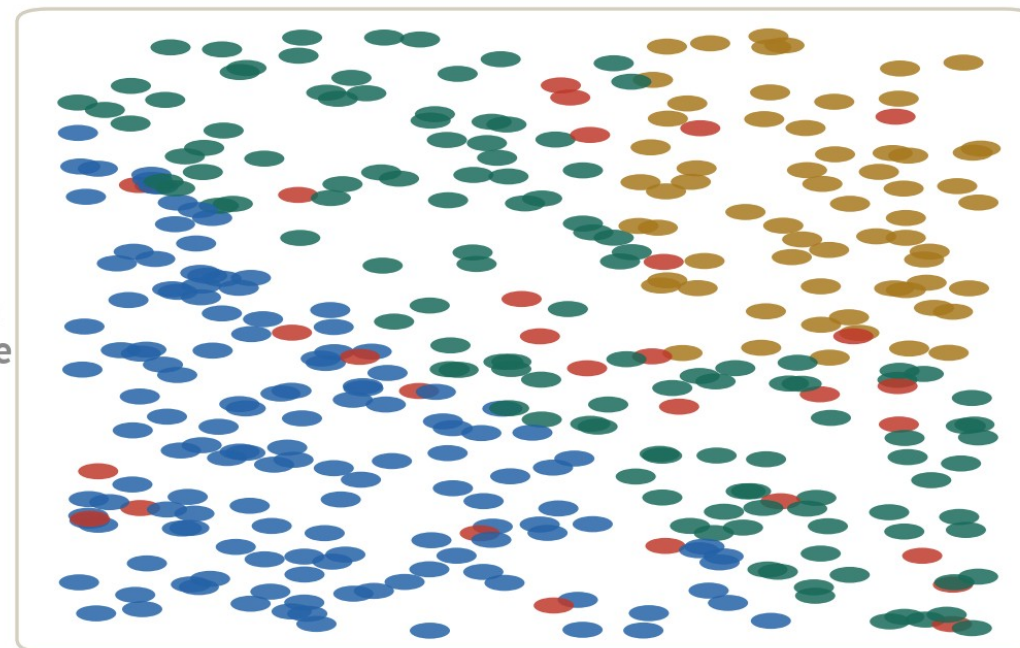
Spot platforms vs single-cell platforms

spot platforms (e.g. Visium)



one spot $\approx$ a mix of 1–10 cells

single-cell platforms (e.g. Xenium)



resolution
vs coverage

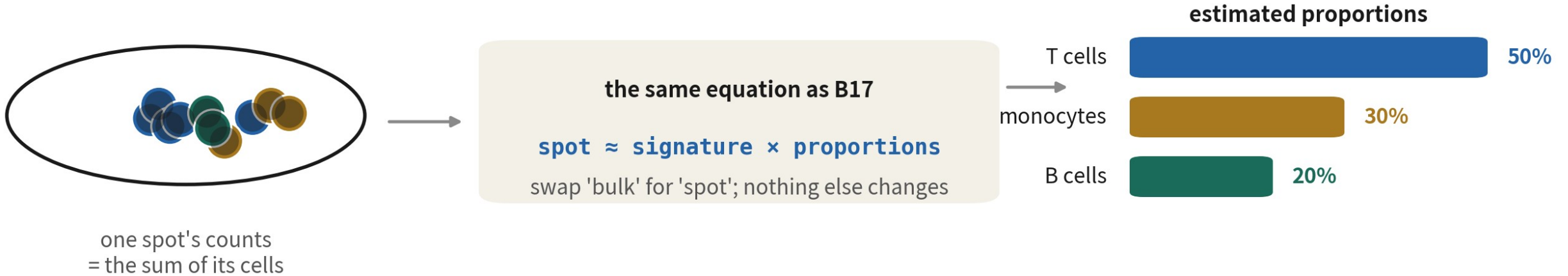
true cells, but a limited gene panel

position is preserved at a price: spots trade resolution, imaging trades gene count

schematic

Spots trade resolution, imaging trades gene count — position is never free

Spot deconvolution: B17's role in space



a single-cell reference decomposes each spot — SPOTlight, RCTD, cell2location all do this

Swap 'bulk' for 'spot' and the signature $\times$ proportions equation carries over untouched

SECTION TAKEAWAY

**The workhorse of spatial analysis
is the toolkit you already built in these 19
episodes.**

QC, normalization, clustering, annotation, deconvolution — new coordinates, same methods.

05

R and Python: the two worlds

A translation table, object anatomy, and a working h5ad round trip

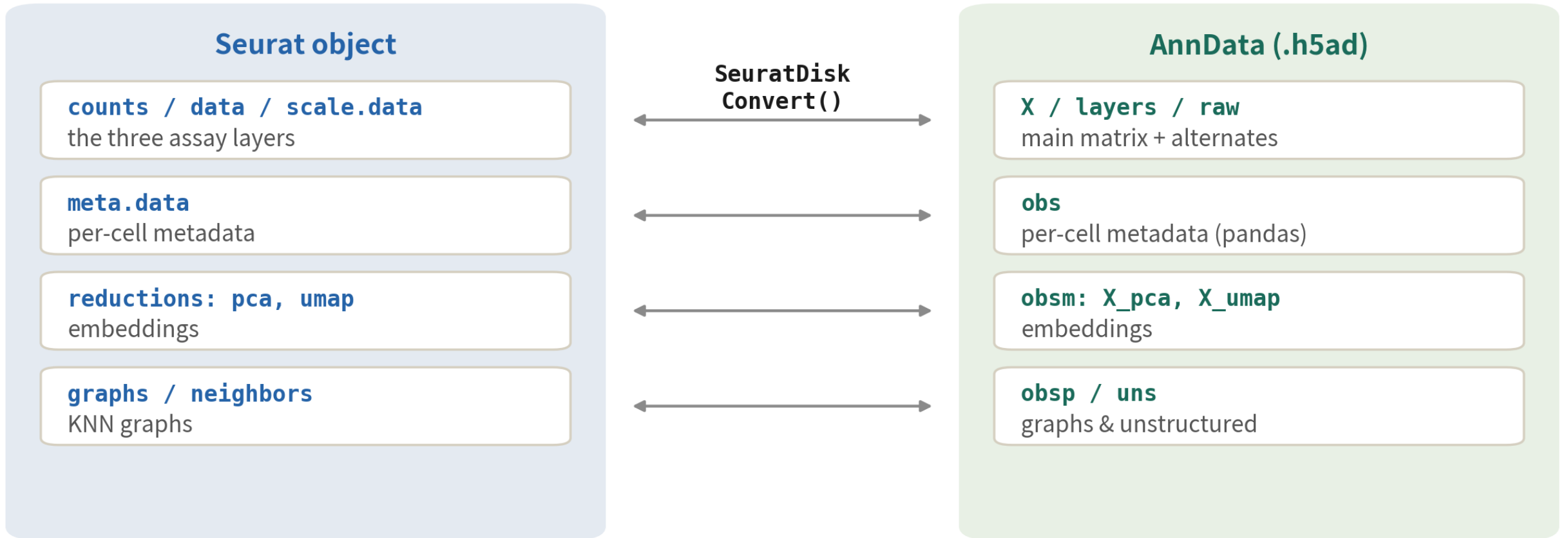
The Seurat ↔ scanpy translation table

step	R • Seurat	Python • scanpy
load 10x	<code>Read10X + CreateSeuratObject</code>	<code>sc.read_10x_mtx</code>
QC metrics	<code>PercentageFeatureSet</code>	<code>sc.pp.calculate_qc_metrics</code>
normalization	<code>NormalizeData</code>	<code>sc.pp.normalize_total + sc.pp.log1p</code>
HVG	<code>FindVariableFeatures</code>	<code>sc.pp.highly_variable_genes</code>
scaling	<code>ScaleData</code>	<code>sc.pp.scale</code>
PCA	<code>RunPCA</code>	<code>sc.tl.pca</code>
neighbours	<code>FindNeighbors</code>	<code>sc.pp.neighbors</code>
clustering	<code>FindClusters</code>	<code>sc.tl.leiden</code>
UMAP	<code>RunUMAP</code>	<code>sc.tl.umap</code>
marker	<code>FindAllMarkers</code>	<code>sc.tl.rank_genes_groups</code>

same pipeline, same concepts — everything from B5–B10 carries over under new names

Every B5–B10 step has a scanpy counterpart — none of the concepts need relearning

Seurat object vs AnnData



key difference: Seurat stores genes × cells; AnnData's X is cells × genes — transposed

assay ↔ X/layers, meta.data ↔ obs, reductions ↔ obsm — and the matrix orientation flips

SaveH5Seurat → Convert

```
# remotes::install_github("mojaveazure/seurat-disk")  
library(SeuratDisk)  
  
pbmc <- readRDS("output/pbmc_b10_annotated.rds") # B10 的成品  
SaveH5Seurat(pbmc, filename = "output/pbmc_b10.h5Seurat")  
Convert("output/pbmc_b10.h5Seurat", dest = "h5ad")  
# 產出 output/pbmc_b10.h5ad
```

Two steps: save h5Seurat, then convert. By default X = the data layer, raw = counts

Unboxing on the Python side

```
in port scanpy as sc
adata = sc.read_h5ad("output/pbm_c_b10.h5ad")
adata
```

```
AnnData object with n_obs x n_vars = 2638 x 13714
  obs: 'orig.ident', 'nCount_RNA', 'nFeature_RNA',
      'percent.mt', 'seurat_clusters', 'celltype'
  obsm : 'X_pca', 'X_umap'
```

Three first looks: n_obs comes first (cells), obs columns complete, obsm has the embeddings

THE CONVERSION RULE

**Conversion is not the end of the job —
it is the start of the inspection.**

Shape, obs, layers — all three checks pass before the file moves on. The trap zone shows what happens otherwise.

06

Public resources and data reuse

Turning other people's data into your control group

Three resources you must know

GEO / SRA



the canonical archive:
matrices from GEO,
FASTQ from SRA

cellxgene census



a standardized h5ad
warehouse, queryable
by tissue and cell type

Azimuth references



the reference family
from B10 —
annotation outsourced
to domain experts

THE REUSE MINDSET

**Your dataset need not fight alone:
public data from the same tissue is a ready-made
control.**

You already have the methods: load, align annotations, integrate and compare — B10 plus B11.

07

Traps: three h5ad conversion pits

Each pit demonstrated for real

Pit 1: sparse matrix orientation

What it is

Seurat stores genes $\times$ cells; AnnData's X is cells $\times$ genes. Manual copies or the wrong slot swap them.



Why it happens

Healthy converters transpose for you, so you never think about orientation — until the day you assemble an object by hand.

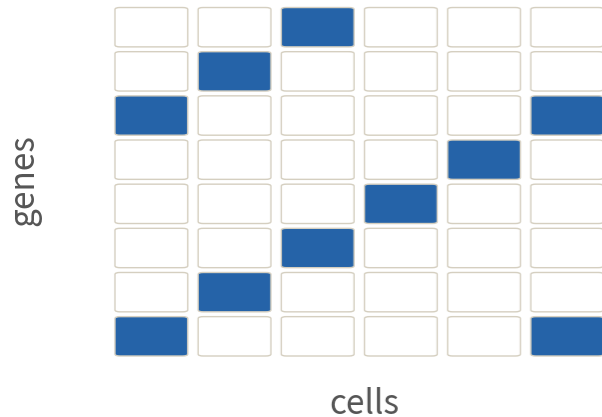
What it costs you

Downstream treats genes as cells: QC metrics nonsense, PCA runs on genes — wildly wrong, no error raised.

Verify orientation at a glance: print the shape

13,714 × 2,638

R / Seurat: genes × cells

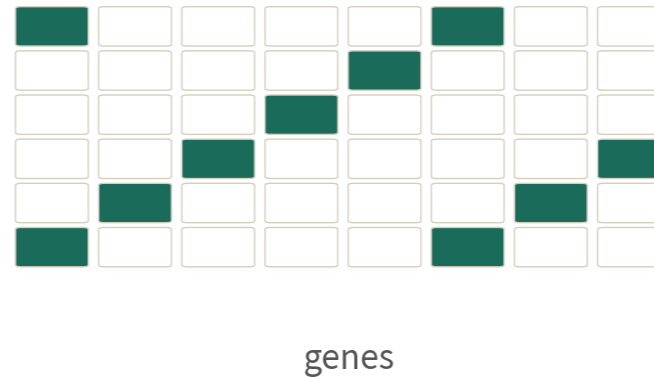


transpose

cells

2,638 × 13,714

Python / AnnData: cells × genes



one-line check

print the shape after
converting: cells first.
Did the numbers swap?
Skipping this is gambling

converters handle the flip when healthy; trouble comes from manual copies and wrong slots

The first line after any conversion prints the shape — cells belong first

Pit 2: metadata column types

What it is

R factors may arrive as plain strings or even numeric codes; level ordering vanishes.



Why it happens

R factors and pandas categories are similar in concept but different in implementation; the bridge makes no guarantee.

What it costs you

Cluster '10' sorts before '2', the control is no longer the first level — legends scramble and model baselines silently change.

Pit 2's check: compare types before and after

```
# R 端：轉出前，逐欄記下 meta 的型別
sapply(pbm c@ meta.data, class)

# Python 端：轉入後，逐欄對照
# adata.obs.dtypes
# factor 應成為 category；掉成 object（字串）就自己補：
# adata.obs["celltype"] = (
#     adata.obs["celltype"].astype("category"))
```

orig.ident	nCount_RNA	nFeature_RNA	percent.m.t
"factor"	"numeric"	"integer"	"numeric"
seurat_clusters	celltype		
"factor"	"character"		

Put the two dtype lists side by side — fix mismatches on the spot, not mid-analysis

Pit 3: lost layers

What it is

scale.data is not carried by default, KNN graphs never convert, extra assays (ADT) need separate handling — all with zero warnings.



Why it happens

The two object models are not isomorphic; the bridge moves what maps cleanly and silently leaves the rest.

What it costs you

Python code treats X as scale.data or clusters on a graph that never arrived — wrong results on data you believed complete.

What survives, what falls off

SaveH5Seurat → Convert("h5ad"): what survives, what falls off

carried over (verify, then use)

- ✓ **data layer → X**
normalized data goes to X
- ✓ **counts → raw**
raw counts stored in raw
- ✓ **meta.data → obs**
per-cell metadata
- ✓ **pca / umap → obsm**
embeddings

dropped (rebuild on the Python side)

- ✗ **scale.data**
not carried by default — recompute
- ✗ **graphs (KNN)**
not converted; rebuild with sc.pp.neighbors
- ✗ **extra assays (ADT...)**
one assay per file; export others separately
- ✗ **factor level order**
may degrade to plain strings

rule: conversion ends with verification — shape, obs columns, layers — not with the file

The right column is not a disaster — it is a to-do list to rebuild on the Python side

The h5ad hand-off checklist

the h5ad hand-off checklist: all six boxes, or it is not done



print the shape: cells $\times$ genes, numbers not swapped



you know whether X is normalized or raw counts



obs columns and dtypes verified (did factors become strings?)



X_pca / X_umap present in obsm



scale.data and KNN graphs: rebuild in Python



extra assays (ADT) handled separately

All six boxes ticked before the hand-off counts — keep this one

TRAPS, SUMMED UP

**Every hand-off between ecosystems
is a house move — inspect the boxes.**

Is the shape right, are the dtypes intact, are the layers all there? Three lines of checks save three days of debugging.

7+

Strategy Room: Complex Samples

The last room — the standard pipeline is done; now imagine the sample in your hands is not PBMC but a tumor

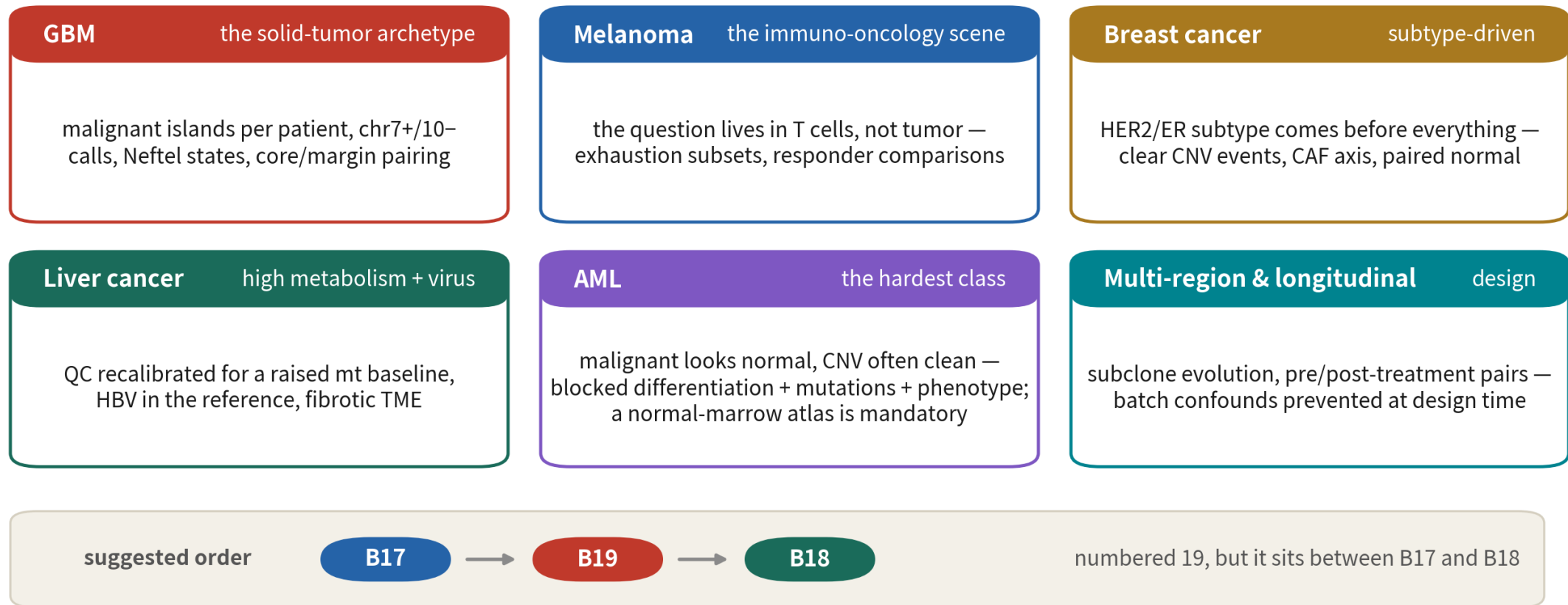
Tools get swapped; decision frameworks live long

Seventeen strategy rooms distil into five decision points — move to scanpy or a spatial platform and every question survives

Decision point	The rule in one sentence	Where it was settled
Who is the statistical unit	the patient, not the cell — open with pseudobulk	B12
How far integration corrects	align the TME; analyse malignant cells per patient	B11
How malignancy is called	CNV with triangulated evidence, never markers alone	B16
How comparisons are designed	paired designs, per-type comparisons, batch prevented at design time	B4 · B12
Which downstream road	check the premise before the tool — curves and networks are inferences	B13–B17

Next stop: B19 — different tumors, different playbooks

B19 • Different tumors, different playbooks — six scenarios



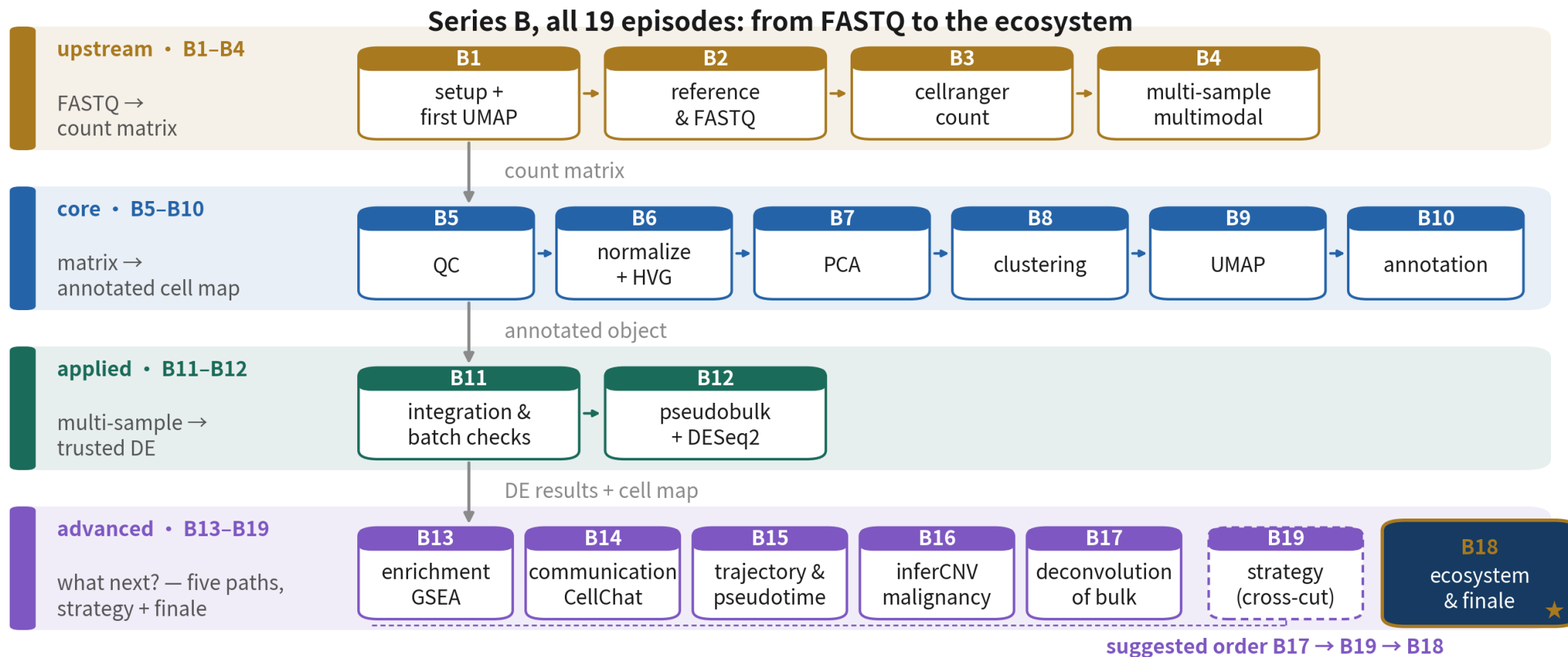
Six scenarios, six playbooks; suggested order B17 → B19 → B18 — the strategy thread continues in B19

08

The series finale

Nineteen episodes, one map

Series B: the full 19-episode map



you are standing at the ★ — you have now walked the entire map

Upstream → core → applied → advanced — you are standing at the star

First half: from FASTQ to a cell map

Upstream B1-B4



FASTQ, references,
count, multi-sample
multimodal — where
data is born

Core B5-B10



QC → normalize →
PCA → cluster →
UMAP → annotate,
end to end

Applied B11-B12



integration aligns
samples; pseudobulk
gets DE right

Second half: after the gene list

B13–B14



enrichment turns lists
into biology; CellChat
infers the conversation

B15–B16



trajectories draw the
time axis; inferCNV
flags malignant cells

B17–B18



deconvolution feeds
bulk cohorts; finally,
the ecosystem laid
open

Three study routes

the maths route

you keep asking: what is this algorithm computing?



Series C: open and derive every black box from B



SVD behind PCA, modularity, UMAP's objective

the engineering route

you keep thinking: how do I automate and reproduce this?



pipelines: targets / Nextflow, containerised environments



Quarto reports: code and conclusions in one document

the biology route

you keep asking: which cell types matter in my tissue?



read deeply: the single-cell atlas of your organ / disease



find matched-tissue data on cellxgene census

the routes are not exclusive — most people walk a bit of each; what matters is not stopping

Maths leads to Series C, engineering to pipelines and Quarto, biology to the literature

ONE-LINE SUMMARY

**What you take away is not nineteen episodes of
commands —
it is the discipline of looking at the data before deciding.**

Commands go stale; discipline does not. Swap in scanpy or a spatial platform — the judgement carries over.

Check yourself 1

At what level are WNN's weights defined?

- A One set shared by the whole dataset
- B One set per cluster
- C One set per cell
- D One set per gene

Answer C. WNN computes weights per cell: the modality whose neighbours better predict that cell's profile gets the higher weight. So ADT settles naive vs memory T while RNA identifies B cells — simultaneously, in the same dataset.

Check yourself 2

On a spot platform like Visium, what is inside one spot?

- A Exactly one cell
- B A mixed signal from roughly 1–10 cells
- C One nucleus
- D The expression of one gene

Answer B. A spot is a fixed capture region covering several cells — spot data is essentially mini-bulk with coordinates, so B17's deconvolution equation applies directly: $\text{spot} \approx \text{signature} \times \text{proportions}$.

Check yourself 3

The h5ad file is written. What should the first line on the Python side do?

- A Run `sc.pp.neighbors` and start analysing
- B Print the object: check shape, obs columns, obsm
- C Plot a UMAP to celebrate
- D Transpose X just in case

Answer B. Conversion starts the inspection: is `n_obs` the cell count, are obs columns and dtypes intact, are `X_pca` / `X_umap` present? Only then proceed — and transposing 'just in case' is precisely how correct orientation gets broken.

SERIES FINALE

Series B: all 19 episodes complete

Thank you for walking the whole map. Next stop, Series C: every black box from Series B opened — from SVD to UMAP's objective, derived step by step.

Tools will update and data will grow — but look first, decide second, write down why remains your opening move.